Optimizing the Flow Shop Problem with SPV Discrete Differential Evolution

Alex Buckley

March 15, 2026

Contents

1 Overview **6**
- 1.1 Experiment . . . 6
- 1.2 The Permutation Flow Shop Problem . . . 6
  - 1.2.1 Blocking Variation . . . 6
  - 1.2.2 The NEH Heuristic . . . 6
- 1.3 Differential Evolution . . . 6

2 Experiment Methodology **7**
- 2.1 DE Implementation . . . 7
  - 2.1.1 Discrete DE . . . 7
- 2.2 Optimization Strategies . . . 7
- 2.3 Flow Shop Inputs . . . 8
  - 2.3.1 Job Times . . . 8
  - 2.3.2 Due Dates . . . 8
  - 2.3.3 DE hyperparameters . . . 9

3 Experiment Results **9**
- 3.1 Makespan Optimization . . . 9
  - 3.1.1 Without Blocking . . . 9
  - 3.1.2 With Blocking . . . 11
- 3.2 Tardiness Optimization . . . 12
  - 3.2.1 Without Blocking . . . 12
  - 3.2.2 With Blocking . . . 14

4 Conclusion **15**

List of Tables

1 Differential Evolution Mutation Strategies 8
2 Differential Evolution Crossover Strategies 8
3 Differential Evolution Hyperparameters 9
4 Algorithm Comparison: Makespan Optimization without Blocking 11
5 Algorithm Comparison: Makespan Optimization with Blocking 12
6 Algorithm Comparison: Tardiness Optimization without Blocking 14
7 Algorithm Comparison: Tardiness Optimization with Blocking 15

List of Figures

1 DE strategy comparison for makespan optimization without blocking 10
2 Execution time comparison for makespan optimization without blocking 10
3 DE strategy comparison for makespan optimization with blocking 11
4 Execution time comparison for makespan optimization with blocking 12
5 DE strategy comparison for tardiness optimization without blocking 13
6 Execution time comparison for tardiness optimization without blocking 13
7 DE strategy comparison for tardiness optimization with blocking 14
8 Execution time comparison for tardiness optimization with blocking 15

List of Equations

1 Initial Population NEH Pertubation 7
2 Original Proportional-Random Method 8
3 Upddated Proportional-Random Method 9

1 Overview

1.1 Experiment

This experiment analyzes the optimization of the flow shop problem with SPV Differential Evolution (DE) optimization. It performs optimizations of the flow shop problem for both makespan and tardiness. Both optimization targets are run with and without the blocking extension. Various DE strategies are used to conduct the optimization, with the NEH heuristic used to provide an initial seed for DE optimization.

1.2 The Permutation Flow Shop Problem

The Permutation Flow Shop Problem involves scheduling jobs across machines in the most efficient manner possible. Inputs include a predefined number of jobs and machines, with each job taking a set amount of time to process on each machine. Jobs must be completed on machines in order, meaning a job must be processed on the first machine before it can be be processed on the second. Job order is adjusted in order to achieve optimal job scheduling.

There are different factors that job order can be optimized for. Firstly there is makespan, the total amount of time it takes for all jobs to be processed on all machines. In order to optimize for tardiness, an additional input *due dates* must be included. Due dates is an array of numbers, with the ith element representing the time at which $jobs_i$ should be completed. For any given job sequence, a *tardiness* array can be created, with $tardiness_i$ representing how long after its due date the ith job finished processing on all machines. If a job finishes before its due date, it has a tardiness value of 0, as tardiness cannot be negative. The total tardiness of a job sequence can be calculated as the sum of each jobs' tardiness values. There are many other aspects of the flow shop problem that can be the target of optimization, however, this experiment only optimizes for makespan and tardiness.

1.2.1 Blocking Variation

This experiment also analyzes optimization for the blocking variation of the flow shop problem. The flow shop blocking problem prevents the use of intermediate job storage between machines. Thus, the blocking variation prevents a job from completing processing on $machine_i$ if $machine_{i+1}$ is busy. Both tardiness and makespan can be the optimization target regardless of whether blocking is used. The blocking variation generally results in increased tardiness and makespan when other inputs are held constant.

1.2.2 The NEH Heuristic

The Nawaz-Enscore-Ham (NEH) heuristic is a constructive algorithm commonly used to find high-quality solutions for the permutation flow shop problem. NEH works by sorting jobs, then testing the insertion of each job at each possible position. The job is then inserted in the most optimal position. Insert position can be chosen based off makespan, tardiness, or any other metric being optimized. However, the initial sort criterion should be adjusted based on the optimization target. When optimizing makespan, this experiment sorts jobs in decreasing order by the total processing time across all machines. Alternatively, tardiness optimization sorts by due date, starting with the earliest due date.

1.3 Differential Evolution

Differential Evolution (DE) is a population-based metaheuristic algorithm designed to solve optimization problems. DE operates by maintaining a population of candidate solutions and iteratively improving them through mutation, crossover, and selection operations. In each generation, a mutant vector is created by adding a weighted difference of randomly selected population members to a base solution, then a trial vector is produced via crossover with the target solution, and the trial vector replaces the target if it achieves a lower objective value. The algorithm's effectiveness stems from its ability to balance exploration and exploitation through control parameters: the mutation scaling factor F controls the magnitude of perturbations, while the crossover rate CR governs the probability of inheriting components from the mutant vector. DE has

proven highly effective across diverse optimization landscapes due to its simplicity, robustness, and minimal hyperparameter requirements compared to other metaheuristics.

2 Experiment Methodology

This experiment analyzes DE optimization results with differing mutation and crossover strategies, for both tardiness and makespan optimization. All experiments are performed with blocking and non-blocking variants. In order to achieve more accurate results, multiple runs are conducted for each experiment configuration, differing only by rng seed. 10 runs are conducted for each problem setup, with the initial seed ranging from 1 to 10. While a higher number of runs, 30, is more standard and reliable for experimentation, the runtime of the experiment was very high, even conducting only 10 runs.

Despite initial seed value always running from 1 to 10 for each problem, actual rng seeds used within the problem throughout the run differ. However, the final seed values used within an experiment is deterministic, based of initial seed and other experiment inputs. This is because each run requires multiple rng generators to be created. Because of the parallelistic nature of this DE implementation, multiple generators are invoked concurrently. Thus, initializing rng objects based on loop iteration plus initial seed is used throughout the experiment in order to avoid race conditions.

2.1 DE Implementation

This experiments utilizes the *OptiPy* library to perform the optimization experiments. Though providing a Python interface, *OptiPy* utilizes *C++* for performing optimizations, as well as OpenMP for CPU-based parallelism. Importantly, *OptiPy* ensures deterministic pseudo-random number generations, allowing for repeatable and consistent experimentation.

Before running the DE optimization, an initial population must be created. While 75% of the initial population is generated pseudo-randomly, an NEH optimization is used to initialize the rest. While the NEH-generated solution is only included once, random noise is added to the remaining initial population, as seen in Eq. (1). When optimizing for makespan $\sigma_p = 0.1$ is used while tardiness optimization has $\sigma_p = 0.02$.

$$\vec{x}_i = \text{clamp}\left(\vec{x}_{\text{NEH}} + \sigma_p \cdot \mathcal{U}(-1, 1), -1, 1\right) \tag{1}$$

2.1.1 Discrete DE

The standard DE algorithm requires a continuous search space in order to function. However, the flow shop problem is inherently combinatorial, thus having a discrete search space. The Standard Permutation Vector (SPV) is used to navigate this discrepancy. SPV allows DE to operate on a continuous search space while still allowing for the generation and evaluation of a job sequence. SPV maps a continuous $\vec{x} \in [-1, 1]^n$ to a job sequence by performing an argsort. This operation outputs a vector of the same length of $\vec{x}$, (n) containing unique integers $\in [0, n-1]$. Thus, an evaluation of makespan or tardiness can be calculated by treating the sorted vector as a job sequence. [2] details the SPV strategy, as well as many other techniques that can be used to apply DE to discrete and combinatorial problems.

2.2 Optimization Strategies

This experiment focuses on the effectiveness of different DE mutation and crossover strategies when applied to the flow shop problem. Utilized Mutations can be seen in Table 1

Table 1: Differential Evolution Mutation Strategies

Strategy	Description
RAND1	$\vec{v} = \vec{x}_r + F(\vec{x}_{r1} - \vec{x}_{r2})$
RAND2	$\vec{v} = \vec{x}_r + F(\vec{x}_{r1} - \vec{x}_{r2}) + F(\vec{x}_{r3} - \vec{x}_{r4})$
BEST1	$\vec{v} = \vec{x}_{best} + F(\vec{x}_{r1} - \vec{x}_{r2})$
BEST2	$\vec{v} = \vec{x}_{best} + F(\vec{x}_{r1} - \vec{x}_{r2}) + F(\vec{x}_{r3} - \vec{x}_{r4})$
RANDTOBEST1	$\vec{v} = \vec{x}_r + K(\vec{x}_{best} - \vec{x}_r) + F(\vec{x}_{r1} - \vec{x}_{r2})$

Table 2 Shows the utilized crossover strategies.

Table 2: Differential Evolution Crossover Strategies

Strategy	Description
BIN (Binomial)	Apply CR probability per dimension; uniform random sampling
EXP (Exponential)	Sequential dimension selection; contiguous block crossover

2.3 Flow Shop Inputs

2.3.1 Job Times

The Taillard benchmark set is a widely-used standard dataset for evaluating permutation flow shop scheduling algorithms, comprising instances with varying numbers of jobs and machines. Each instance in the Taillard set is characterized by its dimensions ($n \times m$), where n is the number of jobs and m is the number of machines, and includes job processing times generated pseudo-randomly to avoid trivial or pathological problem structures. The dataset is valuable for algorithm comparison because known optimal or near-optimal solutions have been established for many instances through extensive prior research, allowing new algorithms to be evaluated against benchmark performance metrics. Taillard instances range from small problems (e.g., 20×5) to large instances (e.g., 100×20), providing a scalability test across problem sizes. This experiment uses Taillard instances to systematically evaluate DE performance on both makespan and tardiness optimization objectives, with and without the blocking constraint, ensuring that results are comparable to prior work and representative of real-world flow shop scheduling complexity. [3] first introduced the Taillard dataset in 1993, and it has been the de facto standard for makespan optimization benchmarking since.

2.3.2 Due Dates

The Taillard dataset does not contain due dates for tardiness calculation. Instead of relying on a data set with due dates, this experiment generates due dates programmatically, with generation partially reliant on pseudo-random number generation. This is done with a seeded instance of NumPy's *default_rng* in order to ensure deterministic and repeatable due date generation.

Due to its prominence in academic research, the *proportional-random method* is used for due date generation. A standard implementation of this method applied to flow shop optimization by [1] is seen in Eq. (2).

$$d_j = \rho \times \left[\sum_{i=1}^{m} (p_{j,i} + st_i) \right] (1 + \text{rand}()) \tag{2}$$

where d_j is the due date for job j, $\rho \in (0, \infty)$ is a tightness factor controlling deadline difficulty, $p_{j,i}$ is the processing time of job j on machine i, st_i is the sequence-independent setup time for machine i, m is the total number of machines, and $\text{rand}() \in [0, 1)$ is a uniformly distributed random variable.

However, this generation method could not be directly implemented in this experiment, as this flowshop model does not track machine setup times. Consequently, we adapted the formula by omitting setup time, as seen in Eq. (3).

$$d_j = \rho \times \left[\sum_{i=1}^{m} p_{j,i} \right] (1 + \text{rand}()) \tag{3}$$

2.3.3 DE hyperparameters

The hyperparameters used in DE optimization can significantly impact the performance of the algorithm. While mutation and crossover strategies are manipulated and analyzed, this experiment uses constant values for other DE inputs. The values, as seen in Table 3, were chosen through informal experimentation. While they are likely far from optimal, they provide consistent DE behavior, allowing the experiment to provide more accurate insight into the effectiveness of different DE mutation and crossover strategies. Additionally, in order to reduce the time required to conduct experiments, population size and max generations were reduced. Optimization results would likely benefit from a higher value for both

Table 3: Differential Evolution Hyperparameters

Parameter	**Value**
Maximum Generations	400
Population Size	200
Mutation Scaling Factor (F)	0.8
Crossover Rate (CR)	0.5

3 Experiment Results

In order to compare DE strategies across differing test cases, results have been averaged. For every combination of mutation and crossover strategy, the mean makespan and tardiness was computed. Mean was calculated seperately for blocking and non-blocking variants, as well as runs optimizing makespan and tardiness. This method helps provide insight of DE strategies across test cases and sizes. However, this methodology likely results in compared values being heavily swayed by test cases with a large number of jobs and machines over smaller cases. Nevertheless, it provides results that can be used to compare DE strategy effectiveness for the flow shop problem.

3.1 Makespan Optimization

3.1.1 Without Blocking

Fig. 1 shows *DE/best-1/exp* and *DE/rand-to-best-1/exp* as the most successful DE variants. Overall, exponential crossover is demonstrated as very successful in optimizing makespan when blocking is not used, as they produced the 4 most successful optimizations. Furthermore *DE/rand-2/exp* and *DE/best-1/exp* tie for the fifth lowest makespan. Thus, the least successful strategy using exponential crossover produced equal results as the most successful binomial crossover strategy.

The relative effectiveness in this experiment of DE mutation strategies remained largely consistent regardless of whether exponential or binomial crossover was used. While *DE/best-1/exp* and *DE/rand-to-best-1/exp* ties for most successful strategies overall, *DE/rand-to-best-1/bin* closely follows *DE/best-1/bin* as the best strategies using binomial crossover. For both crossover methods, *DE/rand-1* and *DE/best-2* produced very similar makespans, with *DE/rand-2* trailing by a larger gap. This suggests the effectiveness of mutation versus crossover strategies are largely independent of each other for this problem.

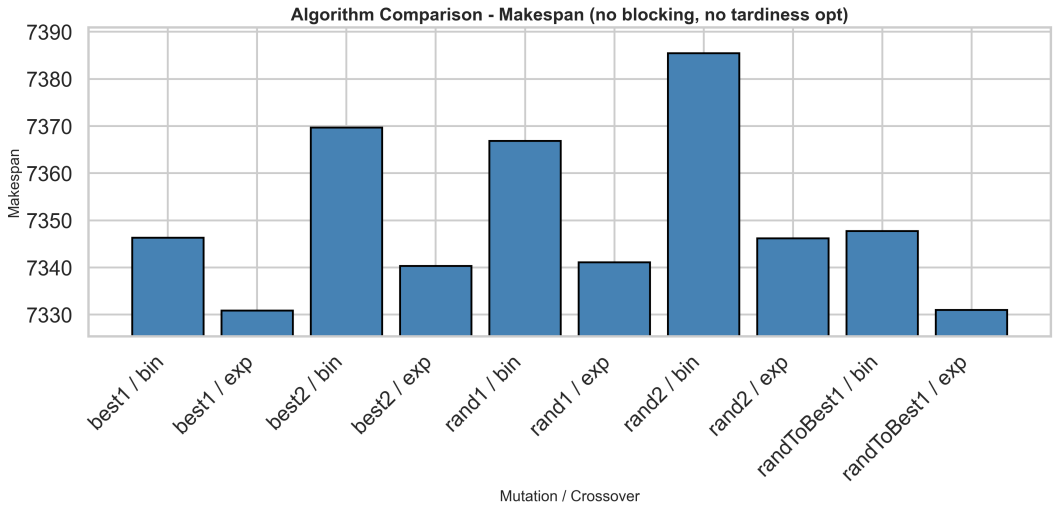

Figure 1: DE strategy comparison for makespan optimization without blocking

Not only did *DE/best-1/exp* provide the best makespan for this problem, Fig. 2 shows it executing significantly quicker than other methodologies. *best-1* seems to perform well in terms of execution speed, as *DE/best-1/bin* holds the second fastest execution time. Overall, exponential crossover seems to perform well in this metric, with *DE/best-1/bin* being the only binomial strategy to be among the 6 fastest algorithms. Exact averaged results can be seen in Table 4.

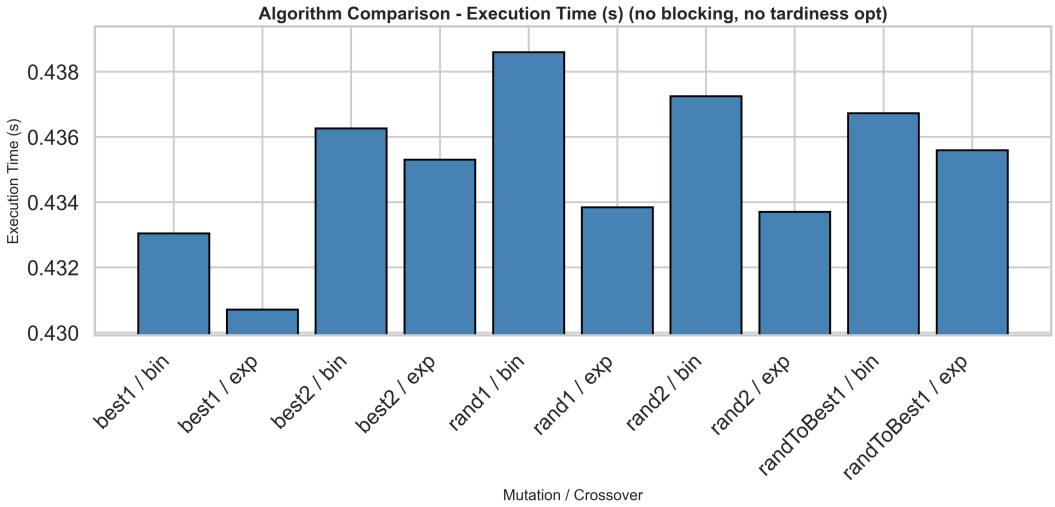

Figure 2: Execution time comparison for makespan optimization without blocking

Table 4: Algorithm Comparison: Makespan Optimization without Blocking

Rank	Algorithm	Makespan	±Std	Tardiness	Time (s)
1	BEST1/EXP	7,331	±0	11,145,703	0.431
2	RANDTOBEST1/EXP	7,331	±0	10,892,308	0.436
3	BEST2/EXP	7,340	±0	10,924,987	0.435
4	RAND1/EXP	7,341	±0	10,942,884	0.434
5	RAND2/EXP	7,346	±0	10,988,387	0.434
6	BEST1/BIN	7,346	±0	11,025,526	0.433
7	RANDTOBEST1/BIN	7,348	±0	10,917,547	0.437
8	RAND1/BIN	7,367	±0	11,038,059	0.439
9	BEST2/BIN	7,370	±0	11,041,468	0.436
10	RAND2/BIN	7,385	±0	11,040,883	0.437

3.1.2 With Blocking

Similarly to the non-blocking variant, *DE/best-1/exp* and *DE/rand-to-best-1/exp* tie for the most successful DE variants, albeit with a higher overall makespan when blocking is used. Furthermore, 3rd and 4th variations trail the top two by a notable margin, with *DE/rand-to-best-1/bin* and *DE/best-1/bin* producing very similar makespans. *DE/rand-2/bin* is again least successful by a significant margin, with *DE/rand-1/bin* and *DE/best-2/bin* placing as 8th and 9th most successful, falling far behind the 7th most effective. These results indicate that while blocking and non-blocking variants produce very different raw makespans, the relative effectiveness of DE strategies remain largely consistent between the two problems. Furthermore, both problems produced aan overall makespan ranges differing by only 1 unit. Table 5 demonstrates this in a visual manner.

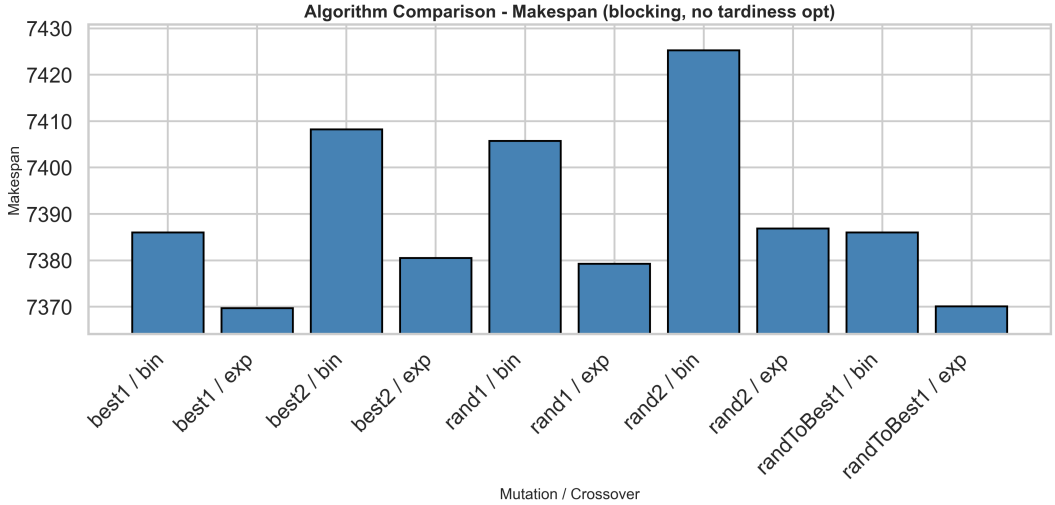

Figure 3: DE strategy comparison for makespan optimization with blocking

Despite the similarity in strategy's relative makespan optimization effectiveness between bloccking and non-blocking problems, execution times differed more significantlly. For example, Fig. 4 demonstrates a *DE/best-1/bin* leading in this metric by a large margin with blocking enabled. However, *best-1* proved to be the fastest mutation strategy in both problems regardless of crossover strategy. While DE/rand-1/bin performed similarly well in both problems, there is little continuity between the execution speed of other strategies

between blocking and non-blocking. While the blocking problem resulted in execution times all within 0.013 seconds of each other, non-blocking produced a much smaller range of 0.008 seconds. Exact averaged experiment results can be seen in Table 5

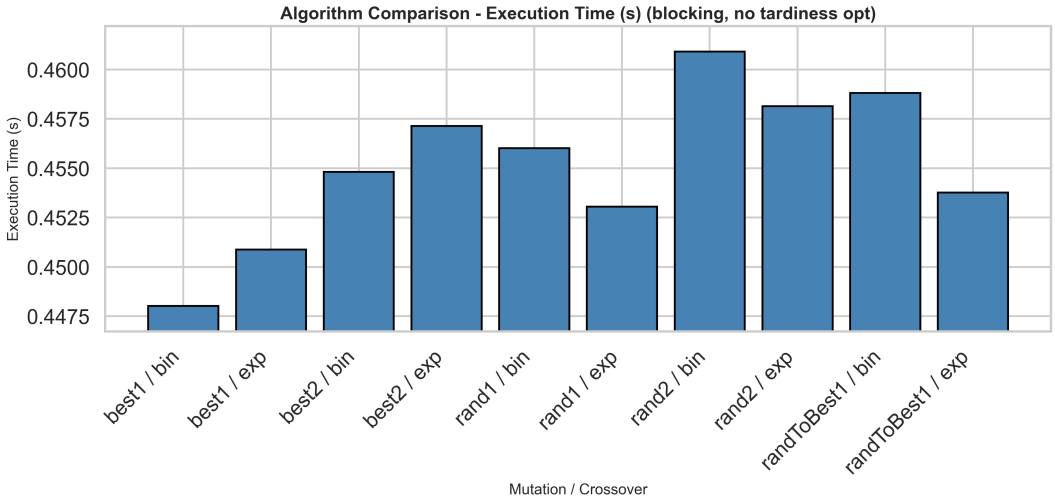

Figure 4: Execution time comparison for makespan optimization with blocking

Table 5: Algorithm Comparison: Makespan Optimization with Blocking

Rank	Algorithm	Makespan	±Std	Tardiness	Time (s)
1	BEST1/EXP	7,370	±0	10,837,772	0.451
2	RANDTOBEST1/EXP	7,370	±0	10,864,002	0.454
3	RAND1/EXP	7,379	±0	10,964,449	0.453
4	BEST2/EXP	7,381	±0	10,939,221	0.457
5	RANDTOBEST1/BIN	7,386	±0	10,929,240	0.459
6	BEST1/BIN	7,386	±0	10,944,476	0.448
7	RAND2/EXP	7,387	±0	10,928,142	0.458
8	RAND1/BIN	7,406	±0	10,989,051	0.456
9	BEST2/BIN	7,408	±0	10,923,764	0.455
10	RAND2/BIN	7,425	±0	10,997,462	0.461

3.2 Tardiness Optimization

3.2.1 Without Blocking

Overall, optimizing for tardiness provided similar results in terms of DE strategies relative effectiveness as optimizing for makespan. For example, Fig. 5 again shows *DE/best-1/exp* and *DE/rand-to-best-1/exp* as the most successful DE variants, followed by similarly performing *DE/rand-1/exp* and *DE/best-2/exp*. *DE/rand-to-best/bin* is again the most successful binomial crossover strategy, with binomial crossover again being strongly outperformed by exponential crossover.

Final averaged tardiness values of each strategy fall in a range of 159,690 units of each other. This range makes up 1.5915% of the most optimal tardiness value. This suggests that tardiness may vary more between strategies than makespan, as for non-blocking makespan optimization, the total makespan range comes out to 0.7463% of the most optimal makespan. In other words, when optimizing for tardiness, the least optimal

strategy's final tardiness was 1.5915% greater than the best, while the worst makespan strategy produced a makespan 0.7463% larger than the best.

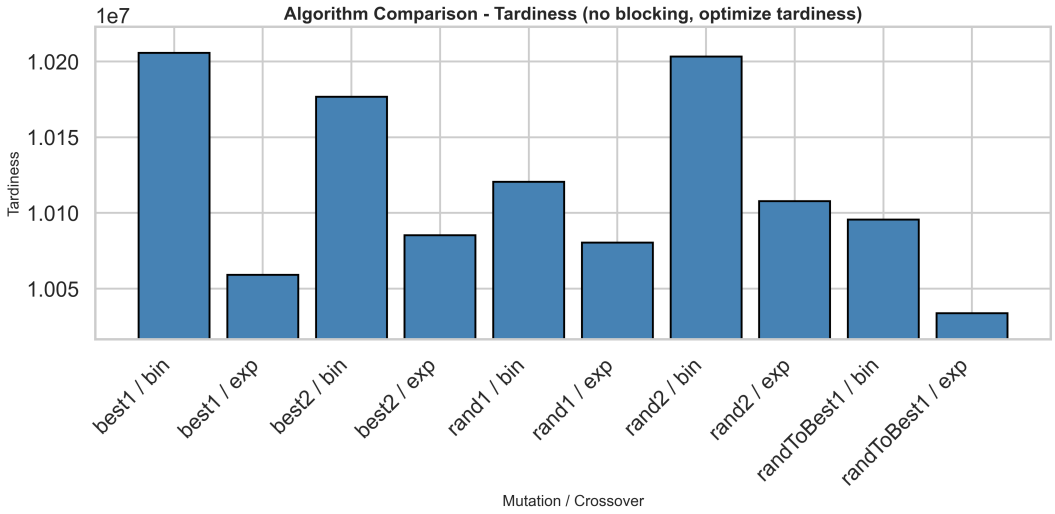

Figure 5: DE strategy comparison for tardiness optimization without blocking

Not only was *DE/rand-to-best-1/exp* the most successful strategy for optimizing tardiness in a non-blocking problem, it also executed the fastest, tied with *DE/best-2/exp*. However, Fig. 6 shows that relative execution time between strategies again differing from previous results. Despite problems producing similar relative optimization results, these discrepancies suggest that execution time is more significantly affected by problem type than optimization success. Execution times for this problem all fall within a range of 0.014 seconds, very similar to the range of times for blocking makespan optimization. Exact averaged results can be seen in Table 6.

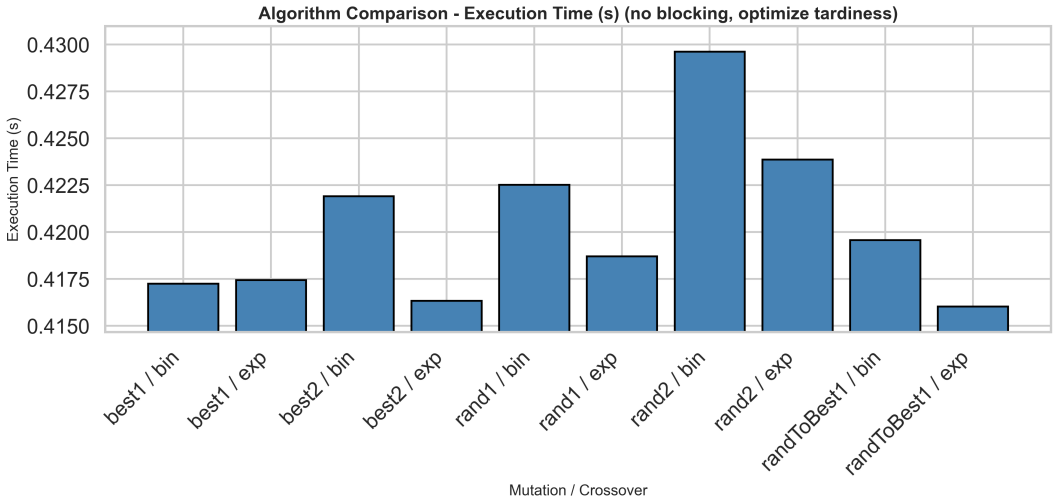

Figure 6: Execution time comparison for tardiness optimization without blocking

Table 6: Algorithm Comparison: Tardiness Optimization without Blocking

Rank	Algorithm	Makespan	±Std	Tardiness	Time (s)
1	RANDTOBEST1/EXP	7,612	±0	10,033,740	0.416
2	BEST1/EXP	7,609	±0	10,059,003	0.417
3	RAND1/EXP	7,621	±0	10,080,381	0.419
4	BEST2/EXP	7,615	±0	10,085,327	0.416
5	RANDTOBEST1/BIN	7,619	±0	10,095,675	0.420
6	RAND2/EXP	7,623	±0	10,107,761	0.424
7	RAND1/BIN	7,637	±0	10,120,438	0.423
8	BEST2/BIN	7,631	±0	10,176,677	0.422
9	RAND2/BIN	7,646	±0	10,203,170	0.430
10	BEST1/BIN	7,613	±0	10,205,641	0.417

When optimizing for tardiness in a blocking problem, *DE/rand-to-best-1/exp* once again proved to be the most successful strategy. Unlike previous problems, however, it led the 2nd best performing by a significant margin. *DE/best-2/exp* performed far better than in previous problems, being the second most successful and leading 3rd by a large gap. Further differing previous problem results, *DE/rand-2/exp* outperformed *DE/rand-1/exp*. However, the overall trend of exponential crossover far outperforming binomial continued, with *DE/rand-to-best-1/bin* once again being the most successful binomial crossover strategy.

Fig. 7 shows the range of total tardiness to be significantly larger in the blocking problem over non-blocking, with a total range of 219,203 units. The least optimal strategy produced a tardiness 2.1976% greater than the most successful strategy, suggesting that blocking produces more variance among results across different DE strategies.

3.2.2 With Blocking

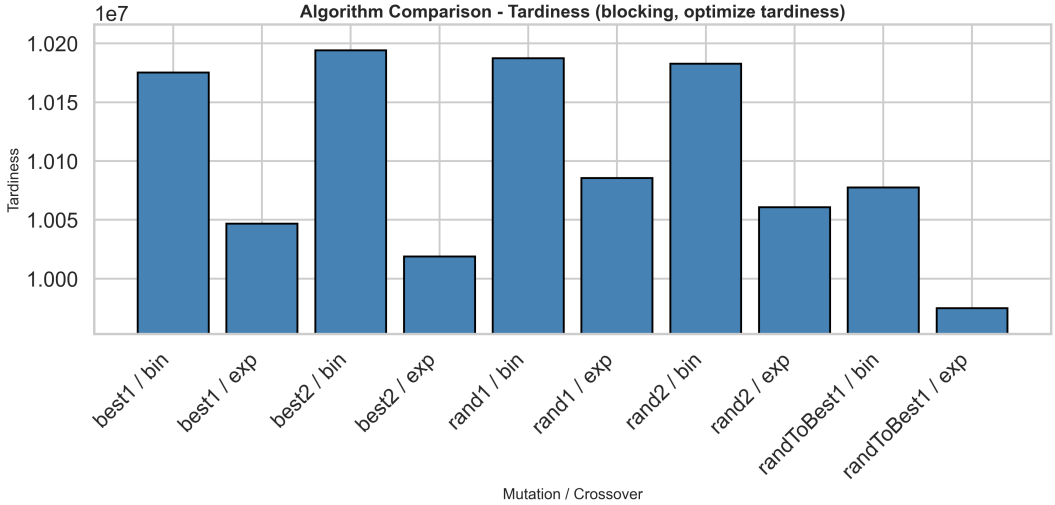

Figure 7: DE strategy comparison for tardiness optimization with blocking

For blocking tardiness optimization, *best-1* appears to be the most computationally efficient strategies, achieving the fastest execution times regardless of mutation strategies. Fig. 8 shows that there is again far less continuity in relative execution times across problem types. Execution times for this problem ranged

by 0.008 seconds, producing a range similar to makespan blocking optimization. The fact that blocking variants had much smaller ranges than non-blocking regardless of optimization metrics suggest that blocking problems may decrease the effect differing mutation and crossover strategies have on execution time. Exact averaged results can be seen in Table 7.

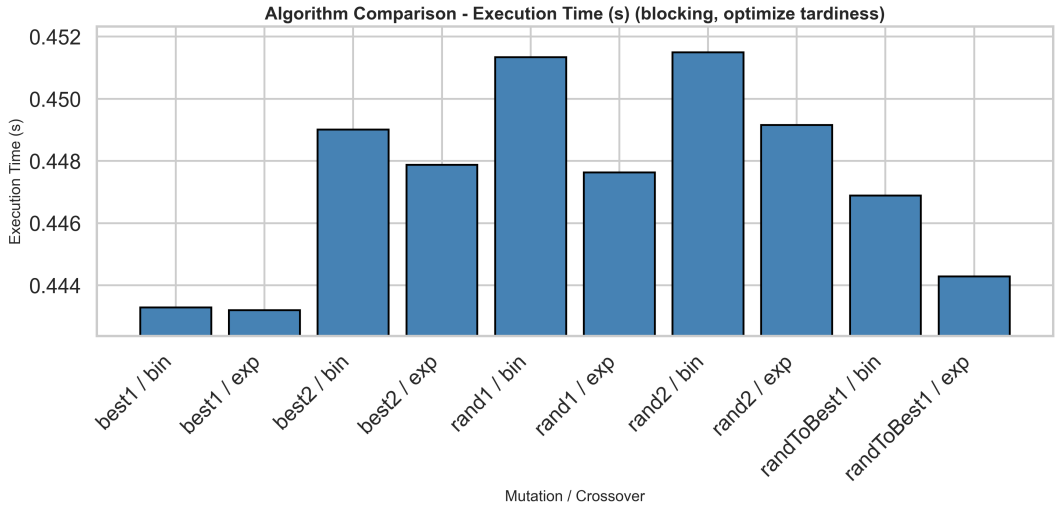

Figure 8: Execution time comparison for tardiness optimization with blocking

Table 7: Algorithm Comparison: Tardiness Optimization with Blocking

Rank	Algorithm	Makespan	±Std	Tardiness	Time (s)
1	RANDTOBEST1/EXP	7,654	±0	9,974,873	0.444
2	BEST2/EXP	7,657	±0	10,018,719	0.448
3	BEST1/EXP	7,648	±0	10,046,868	0.443
4	RAND2/EXP	7,662	±0	10,060,668	0.449
5	RANDTOBEST1/BIN	7,654	±0	10,077,564	0.447
6	RAND1/EXP	7,654	±0	10,085,438	0.448
7	BEST1/BIN	7,647	±0	10,175,235	0.443
8	RAND2/BIN	7,689	±0	10,182,752	0.451
9	RAND1/BIN	7,676	±0	10,187,229	0.451
10	BEST2/BIN	7,670	±0	10,194,076	0.449

4 Conclusion

Overall, DE strategies seemed to retain their relative effectiveness regardless of the optimized metric or the addition of the blocking variation. Several key findings emerged from the analysis.

Strategy Robustness: DE/best-1/exp and DE/rand-to-best-1/exp consistently demonstrated superior performance across all problem variants—makespan and tardiness optimization, with and without blocking. This consistency suggests these mutation strategies are inherently well-suited to the flow shop problem's structure, regardless of the optimization objective.

Crossover Dominance: Exponential crossover substantially outperformed binomial crossover in all tested scenarios. The performance gap ranged from modest (0.75% for non-blocking makespan) to more substantial

(2.2% for blocking tardiness), indicating that the discretization method via Standard Permutation Vector may interact more favorably with exponential crossover's selective inheritance pattern.

Problem Independence: The relative effectiveness of DE strategies remained largely consistent between blocking and non-blocking variants, despite raw objective values differing significantly. This suggests the algorithmic properties that make certain strategies effective are orthogonal to the blocking constraint, offering potential for generalization to other flow shop variants.

Computational Efficiency: Best-1 mutation strategies exhibited faster execution times across most configurations, providing a practical advantage beyond solution quality. However, execution time rankings varied more across problem types than solution quality rankings, suggesting that computational efficiency may be less reliable than solution quality for predicting overall strategy performance.

References

[1] Oualid Elissaouy and Karam Allali. Minimizing the maximum tardiness for a permutation flow shop problem under the constraint of sequence independent setup time. *RAIRO-Operations Research*, 58:377–378, 2024.

[2] Godfrey C. Onwubolu and Donald Davendra. *Differential Evolution: A Handbook for Global Permutation-Based Combinatorial Optimization*. 2009.

[3] Eric Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.